

Analysis and Design of Algorithms

Project II

Problem Statement

Real-World Narrative

A university mentorship program provides weekly one-on-one support sessions between fellows and mentors. Fellows use these sessions to ask for help with technical, academic, or project-related challenges.

Each week, fellows may submit support requests describing what kind of help they need and when they are available. Mentors have limited availability and may only be able to help with certain topics. Because demand changes from week to week and not every request can be satisfied, the program must assign sessions in a way that is fair, explainable, and sensitive to both urgency and long-term waiting time.

The system works with changing information. Some mentor slots may be canceled or rescheduled, and some requests may be withdrawn.

Notation

- F : number of fellows enrolled.
- M : number of mentors.
- S : number of tentative mentor slots offered in a given week.
- R : number of requests submitted in a given week.
- w : current week number (starting from 1).

Input Format (Exact Specification)

All input is read from a single file. All IDs are integers. The file structure is:

Line 1: space-separated fellow IDs (F integers)

Line 2: space-separated mentor IDs (M integers)

Next M lines: For each mentor in the same order as line 2, a line containing:

`mentor_id topic1 topic2 ... topicK`

(Topics are strings without spaces. If a mentor has no topics, write only the mentor ID.)

Then for each week (in order):

- Line: `WEEK w`
- Next line: S (integer)
- Next line: S tokens, each = `mentor_id:time_block` (time block string, e.g., `Mon10`)
- Next line: R (integer)
- Next R lines: each request line with fields separated by spaces:

`fellow_id required_topics_list acceptable_times_list urgency preferred_mentor`
where:

- `required_topics_list`: comma-separated topics (no spaces). Use "" for empty.
- `acceptable_times_list`: comma-separated time blocks.
- `urgency`: one of `high`, `medium`, `low`.
- `preferred_mentor`: integer mentor ID, or 0 if none.

Example:

```
101 102 103
201 202
201 database python
202 networking
```

```
WEEK 1
3
201:Mon10 201:Tue14 202:Mon10
4
101 database,api Mon10 high 0
102 python Mon10 medium 201
103 networking Tue14 low 0
101 database Mon10,Tue14 high 202
```

```
WEEK 2
2
201:Wed11 202:Wed11
3
102 python Wed11 medium 0
103 database Wed11 high 0
101 api Wed11 low 0
```

System State (Pending Requests and Waiting Measure)

The system maintains, for each fellow, the set of **pending requests** – requests that have been submitted (in current or previous weeks) but not yet served (assigned to a slot). When a request is served, it is removed from pending.

For each request r , we know its submission week, denote it by $r.\text{week}$ (derived from the input implicitly). For the current week w , the **age** of request r is $w - r.\text{week}$.

Define the **total backlog waiting time** for fellow f in week w as:

$$W_f(w) = \sum_{r \in \text{pending}(f)} (w - r.\text{week})$$

If a fellow has no pending requests, $W_f(w) = 0$ (empty sum). This measure is used only for end-of-program evaluation, not for weekly decisions.

Simulated Uncertainties

After reading a week’s data, and during assignment, your program must simulate real-world unpredictability:

- Each slot may be canceled or rescheduled with a certain probability. Canceled slots are removed from the available set.
- Each request may be canceled and thus removed from the pending set forever.

These probabilities are not part of the input; you define them in your code. The goal is to test your algorithm’s robustness.

Output Definition

At the end of each week, the system produces:

1. A set of assignments `request_id` $\rightarrow$ `slot_id` where each request is identified by its **index** within the week (1-based order in the input) combined with the week number. Each slot is assigned to at most one request, and each request is assigned to at most one slot. The assignment must satisfy:
 - The slot’s time block is in the request’s acceptable times list.
 - The mentor’s topic set intersects the request’s required topics (at least one topic in common).
2. For every request that remains unserved after assignment, a reason label chosen from:
 - **no feasible slot**: no slot matched availability and topic needs.
 - **lower priority**: a compatible slot existed but was given to higher-priority requests.
 - **request canceled**: the request was randomly canceled before assignment.

Compatibility and Scoring

A request may be assigned to a slot only if:

- The slot's time block is in the request's acceptable times list.
- The mentor's topic set has non-empty intersection with the request's required topics.
- The slot is not canceled.

The **topic match score** for a (request, slot) pair is:

$$\text{topicMatch}(r, s) = |\text{required_topics}(r) \cap \text{mentor_topics}(s)|$$

A score of 0 makes the pair infeasible.

Objective Function (Lexicographic)

Each week, the algorithm must:

1. **Primary goal:** Maximize the number of **requests** that are served (assigned to a slot) this week.
2. **Secondary goal:** Among all assignments that serve the maximum possible number of requests, maximize the total **benefit** of the served requests:

$$\text{Benefit}_{\text{week}} = \sum_{r \in \text{Served}} \left(\alpha \cdot u(\text{urgency}_r) + \beta \cdot \text{topicMatch}(r, \text{slot}_r) + \gamma \cdot p_r + \delta \cdot \text{age}_r \right)$$

where:

- Served is the set of requests assigned this week.
- $u(\text{urgency}_r)$: high = 3, medium = 2, low = 1.
- $\text{topicMatch}(r, \text{slot}_r)$ is defined above.
- $p_r = 1$ if the slot's mentor equals the request's *preferred mentor* (and that mentor is specified and > 0), else 0.
- $\text{age}_r = w - r.\text{week}$ (current week minus submission week).
- $\alpha, \beta, \gamma, \delta$ are positive weights (e.g., $\alpha = 5, \beta = 2, \gamma = 1, \delta = 2$).

End-of-Program Evaluation

After all weeks have been processed, report:

- **Total number of served requests** across all weeks.
- **Total benefit of all served requests** (sum of benefits computed at the time each request was served).

- **Penalty for unserved requests** (optional):

$$\text{Penalty}_{\text{total}} = \sum_{\text{requests } r \text{ that never got served}} \lambda \cdot g(\text{finalAge}_r)$$

where $\text{finalAge}_r = (T + 1) - r.\text{week}$ (the program ends at week T), g is an increasing function (e.g., $g(x) = x^2$), and λ is a penalty factor (e.g., $\lambda = 1$).

Tie-Breaking Rules [Might be removed to simplify things]

When two assignments yield the same number of served requests and the same total benefit score, break ties in this order (applied lexicographically to the sorted list of served requests, e.g., compare the highest-age request first):

1. Higher urgency (high > medium > low) of the request.
2. Larger age ($w - r.\text{week}$) of the request.
3. Higher topic match score $\text{topicMatch}(r, s)$.
4. Preferred mentor match (1 vs 0).
5. Smaller fellow ID (if still tied, smaller request index in the week's input order).

Assumptions

- A fellow may submit multiple requests in a week. There is no per-fellow limit on assignments per week; a fellow could be assigned to multiple slots in the same week if their available time blocks and the slots' timings allow it.
- Slots are one-on-one (capacity 1).
- All input data is known at the start of the week (batch processing). Random cancellations are applied by your program during assignment.
- Topic compatibility requires at least one topic in common (score ≥ 1).
- A request's acceptable times list is fixed at submission and does not change in subsequent weeks. If a fellow's availability changes, they must submit a new request (and may cancel the old one).
- The system aims to reduce request waiting times over time, as reflected in the benefit score and the final penalty.

Sample Cases

Case 1: No Backlog

All requests are new (age = 0). Enough slots exist, all constraints satisfied. **Expected:** All requests served. Primary goal achieved.

Case 2: Age vs Urgency

One slot remains. Two requests compete:

	Request A	Request B
Urgency	medium (2)	high (3)
Age ($w - r.\text{week}$)	5	0
Topic match	2	2

With $\alpha = 5, \delta = 2$: - Benefit A = $5 \cdot 2 + 2 \cdot 5 = 10 + 10 = 20$ - Benefit B = $5 \cdot 3 + 2 \cdot 0 = 15$
Request A is served. This shows that a sufficiently old request can outweigh higher urgency.

Case 3: Multiple Requests from Same Fellow

Fellow C has two pending requests: Request C1 (week 7, age 3) and Request C2 (week 9, age 1). Fellow D has one pending request (week 8, age 2). Enough slots for two requests. The algorithm may serve both C1 and C2 if slots allow, because there is no per-fellow weekly limit. The benefit of serving C1 (age 3) and C2 (age 1) is compared to serving D (age 2) and possibly another request. This tests the request-centric nature.

Successful Solution Criteria

An algorithm's output is considered correct if:

1. All hard constraints are satisfied (time compatibility, topic overlap ≥ 1 , no double assignment of a slot).
2. After applying random cancellations, the assignment is feasible.
3. Unserved requests are assigned a valid reason label.
4. On small instances (e.g., ≤ 10 requests, ≤ 5 slots per week) where exhaustive search is possible, the algorithm achieves the maximum number of served requests.
5. Over multiple random seeds (for cancellations), the algorithm runs without errors and produces reasonable assignments.